

大模型 RAG 经验面

来自：AiGC面试宝典



宁静致远

2024年01月28日 10:12



三

- 大模型 RAG 经验面
 - 一、RAG 基础面
 - 1.1 为什么 大模型 需要 外挂(向量)知识库?
 - 1.2 RAG 思路是怎么样?
 - 1.3 RAG 核心技术是什么?
 - 1.4 RAG prompt 模板 如何构建?
 - 1.5 如何评价RAG项目效果的好坏?
 - 1.6 RAG的检索阶段，常见的向量检索模型有哪些?
 - 1.7 针对通用的RAG，你觉得还有哪些改进点?
 - 二、RAG 优化面
 - 痛点1：文档切分粒度不好把控，既担心噪声太多又担心语义信息丢失
 - 问题描述
 - 问题1：如何让LLM简要、准确回答细粒度知识?
 - 问题2：如何让LLM回答出全面的粗粒度（跨段落）知识?
 - 解决方案
 - 思想（原则）
 - 如何构建关键信息?
 - 常见问题
 - 痛点2：在基于垂直领域 表现不佳
 - 痛点3：langchain 内置 问答分句效果不佳问题
 - 痛点4：如何 尽可能召回与query相关的Document 问题
 - 痛点5：如何让LLM基于query和context得到高质量的response
 - 痛点6：embedding模型在表示text chunks时偏差太大问题
 - 痛点7：不同的 prompt，可能产生完全不同的效果问题
 - 痛点8：llm生成效果问题
 - 痛点9：如何更高质量地召回context喂给llm
 - 三、RAG 工程示例面
 - 3.1 本地知识库问答系统（Langchain-chatGLM）
 - 3.1.1 避坑记录
 - 解决持续网页loading的问题：降低gradio版本，以免高版本检查google字体
 - 解决无法安装detectron2【0517后使用paddleocr代替了】
 - 解决PDF无法加载的问题
 - 解决错误file0
 - 使用paddleocr出现错误
 - 解决moss模型加载错误
 - 解决moss提问出现错误「等待更优解决方案」

一、RAG 基础面

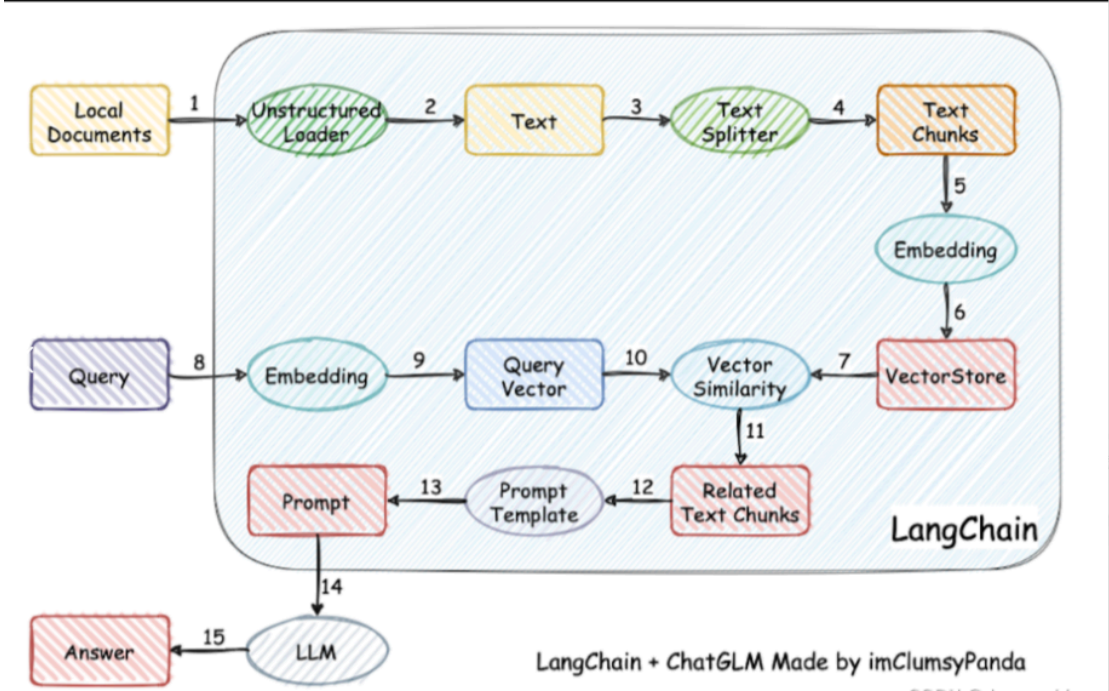
1.1 为什么 大模型 需要 外挂(向量) 知识库？

- 克服遗忘问题
- 提升回答的准确性、权威性、时效性
- 解决通用模型针对一些小众领域没有涉猎的问题
- 提高可控性和可解释性，提高模型的可信度和安全性

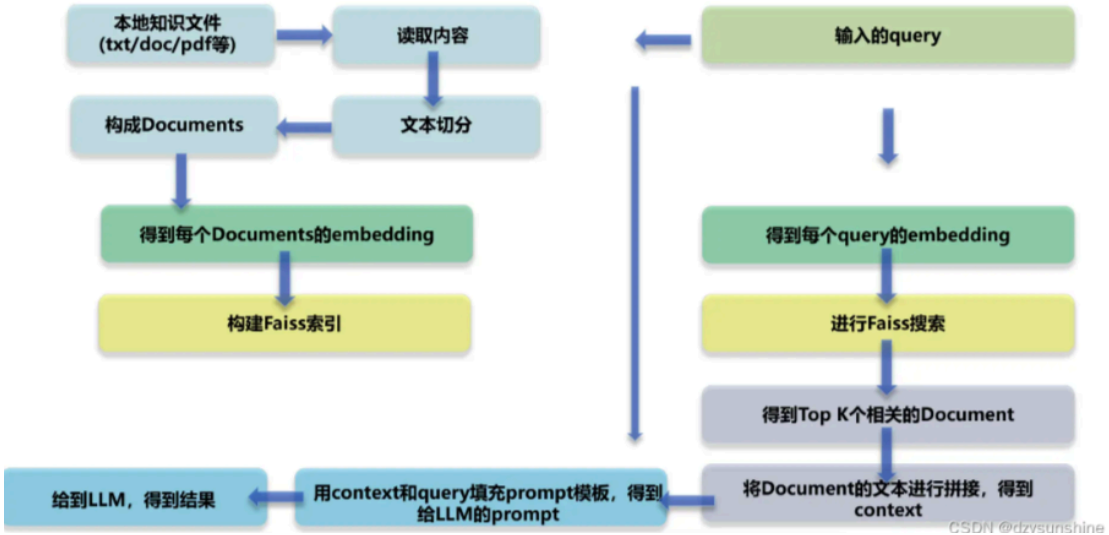
1.2 RAG 思路是怎么样？ 联系小编：yzyykm666

- 1.
1. 加载文件
 2. 读取文本
 3. 文本分割
 4. 文本向量化
 5. 问句向量化
 6. 在文本向量中匹配出与问句向量最相似的 top k 个
 7. 匹配出的文本作为上下文和问题一起添加到 prompt 中
 8. 提交给 LLM 生成回答

版本一



版本二



1.3 RAG 核心技术是什么？关于 NLP 那些你不知道的事

Github: km1994/LLMs interview notes

- RAG 核心技术: embedding 小编: yzyykm666
- 思路: 将用户知识库内容经过 embedding 存入向量知识库, 然后用户每一次提问也会经过 embedding, 利用向量相关性算法 (例如余弦算法) 找到最匹配的几个知识库片段, 将这些知识库片段作为上下文, 与用户问题一起作为 prompt 提交给 LLM 回答

号: 关于 NLP 那些你不知道的事

: km1994/ interview notes

1.4 RAG prompt 模板 如何构建？

已知信息:

{context}

根据上述已知信息, 简洁和专业的来回答用户的问题。如果无法从中得到答案, 请说 “根据已知信息无法回答该问题” 或 “没有提供足够的相关信息”, 不允许在答案中添加编造成分, 答案请使用中文。

问题是: {question}

1.5 如何评价 RAG 项目效果的好坏？

针对检索环节的评估：

- MMR 平均倒排率：查询《或推荐请求》的排名倒数
- Hits Rate 命中率：前 k 项中，包含正确信息的项的数目占比
- NDCG

针对生成环节的评估：

- 非量化：完整性、正确性、相关性
- 量化：Rouge-L

1.6 RAG 的检索阶段，常见的向量检索模型有哪些？

ANN 算法

- 乘积向量
- 暴力搜索
- [hnswlib](#)

KD 树

1.7 针对通用的 RAG，你觉得还有哪些改进点？

- query 侧：做 query 的纠错、改写，规范化和扩展
- 对向量数据库做层次索引，提高检索效率和精度
- 对 LLM 模型微调，针对当前垂直领域引入知识库，提升回答的专业性、时效性和正确性
- 对最终输出做后处理，降低输出的不合理 case

二、RAG 优化面

痛点1：文档切分粒度不好把控，既担心噪声太多又担心语义信息丢失

问题描述

问题1：如何让LLM简要、准确回答细粒度知识？

- 举例及标答如下：

7月17日，国务院新闻办公室举行新闻发布会介绍2023年上半年国民经济运行情况。经初步核算，上半年国内生产总值593034亿元，按不变价格计算，同比增长5.5%，比一季度加快1.0个百分点。分产业看，第一产业增加值30416亿元，同比增长3.7%；第二产业增加值230682亿元，增长4.3%；第三产业增加值331937亿元，增长6.4%。分季度看，一季度国内生产总值同比增长4.5%，二季度增长6.3%。从环比看，二季度国内生产总值增长0.8%。

农业生产形势稳定，畜牧业平稳增长

上半年，农业（种植业）增加值同比增长3.3%。夏粮生产再获丰收。全国夏粮总产量14613万吨，比上年减少127.4万吨，下降0.9%，产量居历史第二高位。上半年，猪牛羊禽肉产量4682万吨，同比增长3.6%，其中猪肉、牛肉、羊肉、禽肉产量分别增长3.2%、4.5%、5.1%、4.3%；牛奶产量增长7.5%，禽蛋产量增长2.9%。二季度末，生猪存栏43517万头，同比增长1.1%。上半年，生猪出栏37548万头，增长2.6%。

用户：2023年我国上半年的国内生产总值是多少？

LLM：根据文档，2023年的国民生产总值是593034亿元。

•需求分析：一是简要，不要有其他废话。二是准确，而不是随意编造。

问题2：如何让LLM回答出全面的粗粒度（跨段落）知识？

• 举例及标签如下：

一、征信中心唯一官方网站为www.pbccrc.org.cn，官方微信公众号为pbc-ccrc。请认准征信中心官方网站域名和官方微信公众号名称，警惕诈骗分子通过短信、网络聊天软件发送的虚假钓鱼网站。

二、征信中心不对外提供逾期记录处理、违规网贷账户关闭、征信修复等相关服务。当您的信用报告出现不良记录时，应及时还款；对信用报告有异议的，可通过人民银行、金融机构等正规渠道依法理性维权。当诈骗分子冒充金融机构或征信中心客服人员，以“注销白条”“注销校园贷”，否则影响个人征信等理由要求注销账户和清空贷款额度时，一定要提高警惕，不要轻信。

三、添加好友共享屏幕是虚假征信类诈骗的常用套路。不轻易下载各类视频聊天软件，不添加私人社交账号与陌生人聊天，更不要共享屏幕进行账户操作，以防账户、存款、贷款、密码等个人信息泄露。

用户：根据文档内容，征信中心有几点声明？

LLM：根据文档内容，有三点声明，分别是：一、.....；二.....；三.....。

需求分析：

要实现语义级别的分割，而不是简单基于 `html` 或者 `pdf` 的换行符分割。

笔者发现目前的痛点是文档分割不够准确，导致模型有可能只回答了两点，而实际上是因为向量相似度召回的结果是残缺的。

有人可能会问，那完全可以把切割粒度大一点，比如每 10 个段落一分。但这样显然不是最优的，因为召回片段太大，噪声也就越多。LLM 本来就有幻觉问题，回答得不会很精准（笔者实测也发现如此）。

所以说，我们的文档切片最好是按照语义切割。

解决方案

思想（原则）

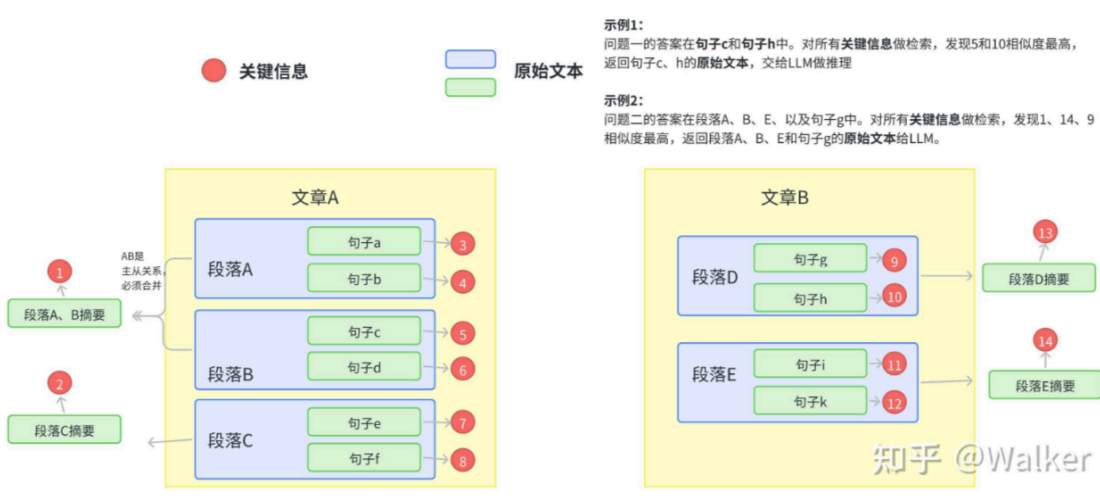
基于 LLM 的文档对话架构分为两部分，先检索，后推理。重心在检索（推荐系统），推理交给 LLM 整合即可。

而检索部分要满足三点 ① 尽可能提高召回率，② 尽可能减少无关信息，③ 速度快。

将所有的文本组织成二级索引，第一级索引是 [关键信息]，第二级是 [原始文本]，二者一一映射。

检索部分只对关键信息做 embedding，参与相似度计算，把召回结果映射的 原始文本 交给 LLM。

主要架构图如下：



如何构建关键信息？

首先从架构图可以看到，句子、段落、文章都要关键信息，如果为了效率考虑，可以不用对句子构建关键信息。

- 1. 文章的切分及关键信息抽取
 - **关键信息:** 为各语义段的关键信息集合，或者是各个子标题语义扩充之后的集合（pdf 多级标题识别及提取见下一篇文章）
 - **语义切分方法 1:** 利用 NLP 的篇章分析（discourse parsing）工具，提取出段落之间的主要关系，譬如上述极端情况 2 展示的段落之间就有从属关系。把所有包含主从关系的段落合并成一段。这样对文章切分完之后保证每一段在说同一件事情。
 - **语义切分方法 2:** 除了 discourse parsing 的工具外，还可以写一个简单算法利用 BERT 等模型来实现语义分割。BERT 等模型在预训练的时候采用了 NSP（next sentence prediction）的训练任务，因此 BERT 完全可以判断两个句子（段落）是否具有语义衔接关系。这里我们可以设置相似度阈值 t，从前往后依次判断相邻两个段落的相似度分数是否大于 t，如果大于则合并，否则断开。当然算法为了效率，可以采用二分法并行判定，模型也不用很大，笔者用 BERT-base-Chinese 在中文场景中就取得了不错的效果。

```
def is_nextsent(sent, next_sent):  
    encoding = tokenizer(sent, next_sent, return_tensors="pt", truncation=True, padding=False)  
  
    with torch.no_grad():  
        outputs = model(**encoding, labels=torch.LongTensor([1]))  
        logits = outputs.logits  
        probs = torch.softmax(logits/TEMPERATURE, dim=1)  
        next_sentence_prob = probs[:, 0].item()  
  
    if next_sentence_prob <= MERGE_RATIO:  
  
        return False  
    else:  
        return True
```

- 1. 语义段的切分及段落（句子）关键信息抽取

如果向量检索效率很高，获取语义段之后完全可以按照真实段落及句号切分，以缓解细粒度知识点检索时大语块噪声多的场景。当然，关键信息抽取笔者还有其他思路。

- 方法 1: 利用 NLP 中的成分句法分析（constituency parsing）工具和命名实体识别（NER）工具提取
 - 成分句法分析（constituency parsing）工具：可以提取核心部分（名词短语、动词短语……）；
 - 命名实体识别（NER）工具：可以提取重要实体（货币名、人名、企业名……）。

譬如说：

原始文本：MM团队的成员都是精英，核心成员是前谷歌高级产品经理张三，前meta首席技术官李四……

关键信息：（MM团队，核心成员，张三，李四）

- 方法 2：可以用语义角色标注（[Semantic Role Labeling](#)）来分析句子的谓词论元结构，提取“谁对谁做了什么”的信息作为关键信息。
- 方法 3：直接法。其实 NLP 的研究中本来就有关键词提取工作（[Keyphrase Extraction](#)）。也有一个成熟工具可以使用。一个工具是 [HanLP](#)，中文效果好，但是付费，免费版调用次数有限。还有一个开源工具是 [KeyBERT](#)，英文效果好，但是中文效果差。
- 方法 4：垂直领域建议的方法。以上两个方法在垂直领域都有准确度低的缺陷，垂直领域可以仿照 [ChatLaw](#) 的做法，即：训练一个生成关键词的模型。[ChatLaw](#) 就是训练了一个 [KeyLLM](#)。

常见问题

- 句子、语义段、之间召回不会有包含关系吗，是否会造成冗余？

回答：会造成冗余，但是笔者试验之后回答效果很好，无论是细粒度知识还是粗粒度（跨段落）知识准确度都比 [Longchain](#) 粗分效果好很多，对这个问题笔者认为可以优化但没必要

痛点 2：在基于垂直领域 表现不佳

模型微调：一个是对 [embedding](#) 模型的基于垂直领域的数据进行微调；一个是对 LLM 模型的基于垂直领域的数据进行微调；

痛点 3：langchain 内置 问答分句效果不佳问题

- 文档加工：
 - 一种是使用更好的文档拆分的方式（如项目中已经集成的达摩院的语义识别的模型及进行拆分）；
 - 一种是改进填充的方式，判断中心句上下文的句子是否和中心句相关，仅添加相关度高的句子；
 - 另一种是文本分段后，对每段分别及进行总结，基于总结内容语义及进行匹配；

痛点 4: 如何 尽可能召回与 query 相关的 Document 问题

- 问题描述: 如何通过得到 query 相关性高的 context, 即与 query 相关的 Document 尽可能多的能被召回;
- 解决方法:
 - 将本地知识切分成 Document 的时候, 需要考虑 Document 的长度、Document embedding 质量和被召回 Document 数量这三者之间的相互影响。在文本切分算法还没那么智能的情况下, 本地知识的内容最好是已经结构化比较好了, 各个段落之间语义关联没那么强。Document 较短的情况下, 得到的 Document embedding 的质量可能会高一些, 通过 Faiss 得到的 Document 与 query 相关度会高一些。
 - 使用 Faiss 做搜索, 前提条件是有高质量的文本向量化工具。因此最好是能基于本地知识对文本向量化工具进行 Finetune。另外也可以考虑将 ES 搜索结果与 Faiss 结果相结合。

痛点 5: 如何让 LLM 基于 query 和 context 得到高质量的 response

- 问题描述: 如何让 LLM 基于 query 和 context 得到高质量的 response
- 解决方法:
 - 尝试多个的 prompt 模版, 选择一个合适的, 但是这个可能有点玄学
 - 用与本地知识问答相关的语料, 对 LLM 进行 Finetune。

痛点 6: embedding 模型在表示 text chunks 时偏差太大问题

- 问题描述:
 1. 一些开源的 embedding 模型本身效果一般, 尤其是当 text chunk 很大的时候, 强行变成一个简单的 vector 是很难准确表示的, 开源的模型在效果上确实不如 openai Embeddings;
 2. 多语言问题, paper 的内容是英文的, 用户的 query 和生成的内容都是中文的, 这里有个语言之间的对齐问题, 尤其是可以用中文的 query embedding 来从英文的 text chunking embedding 中找到更加相似的 top-k 是个具有挑战的问题
- 解决方法:
 1. 用更小的 text chunk 配合更大的 topk 来提升表现, 毕竟 smaller text chunk 用 embedding 表示起来 noise 更小, 更大的 topk 可以组合更丰富的 context 来生成质量更高的回答;
 2. 多语言的问题, 可以找一些更加适合多语言的 embedding 模型

痛点 7: 不同的 prompt, 可能产生完全不同的效果问题

- 问题描述: prompt 是个神奇的东西, 不同的提法, 可能产生完全不同的效果。尤其是指令, 指令型 llm 在训练或者微调的时候, 基本上都有个输出模板, 这个如果前期没有给出 instruction data 说明, 需要做很多的尝试, 尤其是你希望生成的结果是按照一定格式给出的, 需要做更多的尝试

痛点 8: llm 生成效果问题

- 问题描述: llm 本质上是个“接茬”机器, 你给上句, 他补充下一句。但各家的 llm 在理解 context 和接茬这两个环节上相差还是挺多的。最早的时候, 是用一个付费的 gpt 代理作为 llm 来生成内容, 包括解读信息、中文标题和关键词, 整体上来看可读性会强很多, 也可以完全按照给定的格式要求生成相应的内容, 后期非常省心; 后来入手了一台 mac m2, 用 llama.cpp 在本地提供 llm 服务, 模型尝试了 chinese-llama2-alpaca 和 baichuan2, 量化用了 Q6_K level, 据说性能和 fp16 几乎一样, 作为开源模型, 两个表现都还可以。前者是在 llama2 的基础上, 用大量的中文数据进行了增量训练, 然后再用 alpaca 做了指令微调, 后者是开源届的当红炸子鸡。但从 context 的理解上, 两者都比较难像 gpt 那样可以完全准确地生成我希望的格式, baichuan2 稍微好一些。我感觉, 应该是指令微调里自带了一些格式, 所以生成的时候有点“轴”。
- 解决思路: 可以选择一些好玩的开源模型, 比如 llama2 和 baichuan2, 然后自己构造一些 domain dataset, 做一些微调的工作, 让 llm 更听你的话

痛点 9: 如何更高质量地召回 context 喂给 llm

- 问题描述: 初期直接调包 langchain 的时候没有注意, 生成的结果总是很差, 问了很多 Q 给出的 A 乱七八糟的, 后来一查, 发现召回的内容根本和 Q 没啥关系。
- 解决思路: 更加细颗粒度地来做 recall, 当然如果是希望在学术内容上来提升质量, 学术相关的 embedding 模型、指令数据, 以及更加细致和更具针对性的 pdf 解析都是必要的。

参考: PDFTriage: Question Answering over Long, Structured Documents

三、RAG 工程示例面

3.1 本地知识库问答系统 (Langchain-chatGLM)

3.1.1 避坑记录

解决持续网页loading的问题: 降低gradio版本, 以免高版本检查google字体

```
$ pip install gradio==3.21.0
```

解决无法安装detectron2【0517后使用paddleocr代替了】

1. 下载依赖包到环境中: <https://github.com/facebookresearch/detectron2>
2. 然后cd detectron2
3. 注意torch==1.8.0, 安装: pip install -e .
4. 安装完之后将torch升至2.0.0, 同时将protobuf降至3.20.0

解决PDF无法加载的问题

- 确保 apt 包是最新的, 运行 sudo apt update

- 使用 apt 安装 libmagic-dev, poppler-utils 和 tesseract-ocr, 运行 sudo apt install libmagic-dev poppler-utils tesseract-ocr
- 检查 tesseract-ocr 的版本, 运行 tesseract --version
- 检查 tesseract-ocr 的语言包, 运行 tesseract --list-langs
- 如果需要中文识别, 可以在 github 上下载中文识别包2, 并将其放在 /usr/share/tesseract-ocr/4.00/tessdata/ 目录下

解决错误file0

- 按照README里的手工解压pункт和tagger后, 放到指定目录
- пункт.zip 中的 packages/tokenizers 解压, 放到 nltk_data/tokenizers 存储路径下。
- nltk_data 存储路径可以通过 nltk.data.path 查询。
- 将 averaged_perceptron_tagger.zip 下载, 解压放到 nltk_data/taggers 存储路径下
- nltk_data 存储路径可以通过 nltk.data.path 查询。

使用paddleocr出现错误

- 错误: ModuleNotFoundError: No module named 'tools.infer'
- 原因: 在python中import tools能生效, 但是没有相关的包可以卸载, 导致paddleocr中的tools文件无法被引用
- 解决: 需要江paddleocr的文件中, 所有【from tools.infer import】的代码改为: 【from paddleocr.tools.infer import】

解决moss模型加载错误

错误: get_class_from_dynamic_module() missing 2 required positional arguments: 'module_file' and 'class_name'
在2张V100s-32G的环境下, 通过修改以下代码可以解决上面的问题, 供参考: 修改

【langchain-ChatGLM/models/moss_llm.py】文件第33行:

```
def auto_configure_device_map() -> Dict[str, int]:  
    cls = get_class_from_dynamic_module(class_reference="fnlp/moss-moon-003-sft--  
    modeling_moss.MossForCausalLM",  
                                         pretrained_model_name_or_path=llm_model_dict['moss'])
```

为:

```
def auto_configure_device_map() -> Dict[str, int]:  
    cls = get_class_from_dynamic_module(pretrained_model_name_or_path="fnlp/moss-moon-003-sft",  
                                         module_file="modeling_moss.py", class_name="MossForCausalLM")
```

且保证moss的模型文件放置在【angchain-ChatGLM/fnlp/moss-moon-003-sft】路径下, 可以成功加载模型:

Explicitly passing a 'revision' is encouraged when loading a model with custom code to ensure no malicious code has been contributed in a newer revision.
Explicitly passing a 'revision' is encouraged when loading a configuration with custom code to ensure no malicious code has been contributed in a newer revision.
Explicitly passing a 'revision' is encouraged when loading a model with custom code to ensure no malicious code has been contributed in a newer revision.
Explicitly passing a 'revision' is encouraged when loading a configuration with custom code to ensure no malicious code has been contributed in a newer revision.
Overriding torch_dtype=None with 'torch_dtype=torch.float16' due to requirements of 'bitsandbytes' to enable model loading in mixed int8. Either pass torch_dtype=torch.float16 or don't pass this argument at all to remove this warning.

BUG REPORT

Welcome to bitsandbytes. For bug reports, please run
python -m bitsandbytes
and submit this information together with your error trace to: <https://github.com/TimDettmers/bitsandbytes/issues>

bin /usr/local/lib/python3.8/site-packages/bitsandbytes/libbitsandbytes_cuda117_nocublas12.so
CUDA SETUP: CUDA runtime path found: /usr/local/cuda-11.7/lib64/libcudart.so
CUDA SETUP: Highest compute capability among GPUs detected: 7.0
CUDA SETUP: Detected CUDA version 117
CUDA SETUP: Loading binary /usr/local/lib/python3.8/site-packages/bitsandbytes/libbitsandbytes_cuda117_nocublas12.so...
Loading checkpoint shards: 100%
INFO 2023-05-17 02:15:12,188-Id: Load pretrained SentenceTransformer: GanymedNll/text2vec-large-chinese 4/4 [02:00-00:00, 36.19x/it]
WARNING 2023-05-17 02:15:12,188-Id: No sentence-transformers model found with name GanymedNll/text2vec-large-chinese. Creating a new one with MEAN pooling.
Input your local knowledge file path 请输入本地知识文件路径: /workspace/code/langchain-ChatGLM/docs/2018年政策资金.PDF [2023/05/17 02:15:39] ppo: WARNING: Since the angle classifier is not initialized, the angle classifier will not be used during the forward process.
INFO 2023-05-17 02:16:42,231-Id: 2018年政策资金.PDF 已成功加载
INFO 2023-05-17 02:16:42,231-Id: 文件加载完毕, 正在生成向量
ExitCode: 100 [24/24 [00:03-00:00, 7.65it/s]
INFO 2023-05-17 02:16:45,724-Id: Loading faiss with AVX2 support.
INFO 2023-05-17 02:16:45,748-Id: Successfully loaded faiss with AVX2 support.
Input your question 请输入问题: 科技奖励支持计划的对象是谁

Every 1.0s: notebook-rn-202305175348850360-361p4-0: Wed May 17 02:15:39 2023
Wed May 17 02:15:39 2023

NVIDIA-SMI 470.57.02		Driver Version: 470.57.02		CUDA Version: 11.4	
GPU Name	Persistence-M	Bus-Id	Disp-A	Volatile Uncorr. ECC	GPU-Util
Fan Temp	Perf	PwrUsage/Cap	Memory-Usage	Compute M.	MIG M.
0 Tesla V100S-PCI...	Off	00000000:1A:00:0	Off	0	0
N/A 36C P0	30W / 250W	5373MiB / 32510MiB	0%	Default	N/A
1 Tesla V100S-PCI...	Off	00000000:1E:00:0	Off	0	0
N/A 36C P0	37W / 250W	3793MiB / 32510MiB	0%	Default	N/A

Processes:
GPU CI ID PID Type Process name GPU Memory Usage

知乎 @袋鼠猪

解决moss提问出现错误「等待更优解决方案」

- 错误: RuntimeError: probability tensor contains either inf, nan or element < 0
- 解决:
 - emove 【do_sample=True】 can pass the error
 - refer:THUDM/ChatGLM-6B#31 (comment)
 - 不报错, 但是会卡很久才回答

```
CUDA SETUP: Loading binary /usr/local/lib/python3.8/site-packages/bitsandbytes/libbitsandbytes_cuda117_nocublaslt.so...
Loading checkpoint shards: 100% | 4/4 [00:25<00:00, 6.45s/it]
INFO 2023-05-17 03:25:37,699-Id: Load pretrained SentenceTransformer: GanymedeNil/text2vec-large-chinese
WARNING 2023-05-17 03:25:37,699-Id: No sentence-transformers model found with name GanymedeNil/text2vec-large-chinese. Creating a new one with MEAN pooling
Input your local knowledge file path 请输入本地知识文件路径: /workspace/code/langchain-ChatGLM/docs/2018年政策资金.PDF
[2023/05/17 03:32:55] ppoctr WARNING: Since the angle classifier is not initialized, the angle classifier will not be used during the forward process
INFO 2023-05-17 03:32:58,100-Id: 2018年政策资金.PDF 已成功加载
INFO 2023-05-17 03:32:58,100-Id: 文件加载完毕, 正在生成向量库
Batches: 100% | 24/24 [00:03<00:00, 7.69it/s]
INFO 2023-05-17 03:33:01,657-Id: Loading faiss with AVX2 support.
INFO 2023-05-17 03:33:01,683-Id: Successfully loaded faiss with AVX2 support.
Input your question 请输入问题: 科技奖励支持计划的对象是谁
Batches: 100% | 1/1 [00:00<00:00, 36.84it/s]
Setting 'pad_token_id' to 'eos_token_id':106068 for open-end generation.
Traceback (most recent call last):
  File "/workspace/code/langchain-ChatGLM/cli_demo.py", line 48, in <module>
    45
    46
    47 if __name__ == "__main__":
> 48 |     main()
    49
  File "/workspace/code/langchain-ChatGLM/cli_demo.py", line 30, in main
    27 while True:
    28     query = input("Input your question 请输入问题: ")
    29     last_print_len = 0
> 30     for resp, history in local_doc_qa.get_knowledge_based_answer(query=query,
    31                                                                    vs_path=vs_path,
    32                                                                    chat_history=history,
    33                                                                    streaming=STREAMING):
  File "/workspace/code/langchain-ChatGLM/chains/local_doc_qa.py", line 262, in get_knowledge_based_answer
    259         torch_gc()
    260         prompt = generate_prompt(related_docs_with_score, query)
    261
> 262         for result, history in self.llm_call(prompt=prompt,
    263                                              history=chat_history,
    264                                              streaming=streaming):
    265             torch_gc()
  File "/workspace/code/langchain-ChatGLM/models/moss_llm.py", line 85, in _call
    82         inputs = self.tokenizer(prompt_w_history, return_tensors="pt")
    83         with torch.no_grad():
> 85             outputs = self.model.generate(
    86                 inputs.input_ids.cuda(),
    87                 attention_mask=inputs.attention_mask.cuda(),
    88                 max_length=self.max_token,
  File "/usr/local/lib/python3.8/site-packages/torch/utils/_contextlib.py", line 115, in decorate_context
    112 @functools.wraps(func)
    113 def decorate_context(*wargs, **kwargs):
    114     with ctx_factory():
> 115         return func(*wargs, **kwargs)
    116
    117     return decorate_context
    118
  File "/usr/local/lib/python3.8/site-packages/transformers/generation/utils.py", line 1452, in generate
    1449         )
    1450
    1451         # 13. run sample
> 1452         return self.sample(
    1453             input_ids,
    1454             logits_processor=logits_processor,
    1455             logits_warper=logits_warper,
  File "/usr/local/lib/python3.8/site-packages/transformers/generation/utils.py", line 2504, in sample
    2501         # sample
    2502         probs = nn.functional.softmax(next_token_scores, dim=-1)
    2503         next_tokens = torch.multinomial(probs, num_samples=1).squeeze(1)
> 2504
    2505         # finished sentences should have their next token be a padding token
    2506         if eos_token_id is not None:
    2507             if eos_token_id in next_tokens:
RuntimeError: probability tensor contains either 'inf', 'nan' or element < 0
root@notebook-rr-20230516175348850360-361p4-0:/workspace/code/langchain-ChatGLM#
```

知乎 @袋鼠猪